

Module

Import math module

In [1]: 25

Out[1]: 25

In [2]: `x = sqrt(25)` *# 'sqrt' is inbuilt function*

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[2], line 1  
----> 1 x = sqrt(25)    # 'sqrt' is inbuilt function  
  
NameError: name 'sqrt' is not defined
```

In [3]: `import math` *# 'math' is module*

In [4]: `x = math.sqrt(25)`
x

Out[4]: 5.0

In [5]: `x1 = math.sqrt(15)`
x1

Out[5]: 3.872983346207417

In [6]: `import math`
`math.sqrt(25)`

Out[6]: 5.0

In [7]: `math.floor(3.5)` *# floor - Minimum or Least value*

Out[7]: 3

In [8]: `math.ceil(3.5)` *# ceil - Maximum or highest value*

Out[8]: 4

In [9]: `print(math.floor(2.9))`

2

In [10]: `print(math.ceil(2.9))`

3

In [11]: `print(math.pow(2,3))` *# pow - 2 to the power 3*

8.0

```
In [12]: print(math.pow(3,2))
```

9.0

```
In [13]: print(math.pi)    # These are constant
```

3.141592653589793

```
In [14]: print(math.e)    # These are constant
```

2.718281828459045

```
In [15]: m.floor(3.4)
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[15], line 1  
----> 1 m.floor(3.4)  
  
NameError: name 'm' is not defined
```

```
In [16]: import math as m    # Here, we define 'm' as math module  
         print(m.floor(3.4))
```

3

```
In [17]: print(m.ceil(3.4))
```

4

```
In [18]: print(m.sqrt(144))
```

12.0

```
In [19]: # 'math' has many function if you want to call specific function then you can us  
  
         from math import sqrt, pow, floor, ceil # Here only 4 functions will work  
         print(pow(2,3))  
         print(sqrt(25))  
         print(floor(2.3))  
         print(ceil(2.3))
```

8.0

5.0

2

3

```
In [20]: round(pow(2,3))
```

Out[20]: 8

```
In [21]: help()
```

Welcome to Python 3.14's help utility! If this is your first time using Python, you should definitely check out the tutorial at <https://docs.python.org/3.14/tutorial/>.

Enter the name of any module, keyword, or topic to get help on writing Python programs and using Python modules. To get a list of available modules, keywords, symbols, or topics, enter "modules", "keywords", "symbols", or "topics".

You can use the following keyboard shortcuts at the main interpreter prompt. F1: enter interactive help, F2: enter history browsing mode, F3: enter paste mode (press again to exit).

Each module also comes with a one-line summary of what it does; to list the modules whose name or summary contain a given string such as "spam", enter "modules spam".

To quit this help utility and return to the interpreter, enter "q", "quit" or "exit".

Help on built-in module math:

NAME

math

DESCRIPTION

This module provides access to the mathematical functions defined by the C standard.

FUNCTIONS

`acos(x, /)`

Return the arc cosine (measured in radians) of x.

The result is between 0 and pi.

`acosh(x, /)`

Return the inverse hyperbolic cosine of x.

`asin(x, /)`

Return the arc sine (measured in radians) of x.

The result is between -pi/2 and pi/2.

`asinh(x, /)`

Return the inverse hyperbolic sine of x.

`atan(x, /)`

Return the arc tangent (measured in radians) of x.

The result is between -pi/2 and pi/2.

`atan2(y, x, /)`

Return the arc tangent (measured in radians) of y/x.

Unlike `atan(y/x)`, the signs of both x and y are considered.

`atanh(x, /)`

Return the inverse hyperbolic tangent of x.

`cbrt(x, /)`

Return the cube root of x.

`ceil(x, /)`

Return the ceiling of x as an Integral.

This is the smallest integer $\geq x$.

`comb(n, k, /)`

Number of ways to choose k items from n items without repetition and with out order.

Evaluates to $n! / (k! * (n - k)!)$ when $k \leq n$ and evaluates to zero when $k > n$.

Also called the binomial coefficient because it is equivalent to the coefficient of k-th term in polynomial expansion of the expression $(1 + x)^n$.

Raises `TypeError` if either of the arguments are not integers.

Raises `ValueError` if either of the arguments are negative.

`copysign(x, y, /)`
 Return a float with the magnitude (absolute value) of x but the sign of y.

On platforms that support signed zeros, `copysign(1.0, -0.0)` returns -1.0.

`cos(x, /)`
 Return the cosine of x (measured in radians).

`cosh(x, /)`
 Return the hyperbolic cosine of x.

`degrees(x, /)`
 Convert angle x from radians to degrees.

`dist(p, q, /)`
 Return the Euclidean distance between two points p and q.

The points should be specified as sequences (or iterables) of coordinates. Both inputs must have the same dimension.

Roughly equivalent to:
`sqrt(sum((px - qx) ** 2.0 for px, qx in zip(p, q)))`

`erf(x, /)`
 Error function at x.

`erfc(x, /)`
 Complementary error function at x.

`exp(x, /)`
 Return e raised to the power of x.

`exp2(x, /)`
 Return 2 raised to the power of x.

`expm1(x, /)`
 Return $\exp(x)-1$.

This function avoids the loss of precision involved in the direct evaluation of $\exp(x)-1$ for small x.

`fabs(x, /)`
 Return the absolute value of the float x.

`factorial(n, /)`
 Find n!.

`floor(x, /)`
 Return the floor of x as an Integral.

This is the largest integer $\leq x$.

`fma(x, y, z, /)`
 Fused multiply-add operation.

Compute $(x * y) + z$ with a single round.

`fmod(x, y, /)`

Return `fmod(x, y)`, according to platform C.

`x % y` may differ.

`frexp(x, /)`

Return the mantissa and exponent of `x`, as pair `(m, e)`.

If `x` is a finite nonzero number, then `m` is a float with $0.5 \leq \text{abs}(m) < 1.0$ and an integer `e` is such that `x == m * 2**e` exactly. Else, return `(x, 0)`.

`fsum(seq, /)`

Return an accurate floating-point sum of values in the iterable `seq`.

Assumes IEEE-754 floating-point arithmetic.

`gamma(x, /)`

Gamma function at `x`.

`gcd(*integers)`

Greatest Common Divisor.

`hypot(*coordinates)`

Multidimensional Euclidean distance from the origin to a point.

Roughly equivalent to:

`sqrt(sum(x**2 for x in coordinates))`

For a two dimensional point `(x, y)`, gives the hypotenuse using the Pythagorean theorem: `sqrt(x*x + y*y)`.

For example, the hypotenuse of a 3/4/5 right triangle is:

```
>>> hypot(3.0, 4.0)
5.0
```

`isclose(a, b, *, rel_tol=1e-09, abs_tol=0.0)`

Determine whether two floating-point numbers are close in value.

`rel_tol`

maximum difference for being considered "close", relative to the magnitude of the input values

`abs_tol`

maximum difference for being considered "close", regardless of the magnitude of the input values

Return True if `a` is close in value to `b`, and False otherwise.

For the values to be considered close, the difference between them must be smaller than at least one of the tolerances.

`-inf`, `inf` and `NaN` behave similarly to the IEEE 754 Standard. That is, `NaN` is not close to anything, even itself. `inf` and `-inf` are only close to themselves.

`isfinite(x, /)`

Return True if `x` is neither an infinity nor a NaN, and False otherwise.

`isinf(x, /)`

Return True if x is a positive or negative infinity, and False otherwise.

`isnan(x, /)`

Return True if x is a NaN (not a number), and False otherwise.

`isqrt(n, /)`

Return the integer part of the square root of the input.

`lcm(*integers)`

Least Common Multiple.

`ldexp(x, i, /)`

Return $x * (2^{**i})$.

This is essentially the inverse of `frexp()`.

`lgamma(x, /)`

Natural logarithm of absolute value of Gamma function at x.

`log(...)`

`log(x, [base=math.e])`

Return the logarithm of x to the given base.

If the base is not specified, returns the natural logarithm (base e) of x.

`log10(x, /)`

Return the base 10 logarithm of x.

`log1p(x, /)`

Return the natural logarithm of 1+x (base e).

The result is computed in a way which is accurate for x near zero.

`log2(x, /)`

Return the base 2 logarithm of x.

`modf(x, /)`

Return the fractional and integer parts of x.

Both results carry the sign of x and are floats.

`nextafter(x, y, /, *, steps=None)`

Return the floating-point value the given number of steps after x towards y.

If steps is not specified or is None, it defaults to 1.

Raises a `TypeError`, if x or y is not a double, or if steps is not an integer. Raises `ValueError` if steps is negative.

`perm(n, k=None, /)`

Number of ways to choose k items from n items without repetition and with order.

Evaluates to $n! / (n - k)!$ when $k \leq n$ and evaluates to zero when $k > n$.

If k is not specified or is None, then k defaults to n and the function returns n!.

Raises `TypeError` if either of the arguments are not integers.
Raises `ValueError` if either of the arguments are negative.

`pow(x, y, /)`
Return `x**y` (x to the power of y).

`prod(iterable, /, *, start=1)`
Calculate the product of all the elements in the input iterable.

The default start value for the product is 1.

When the iterable is empty, return the start value. This function is intended specifically for use with numeric values and may reject non-numeric types.

`radians(x, /)`
Convert angle x from degrees to radians.

`remainder(x, y, /)`
Difference between x and the closest integer multiple of y.

Return `x - n*y` where `n*y` is the closest integer multiple of y. In the case where x is exactly halfway between two multiples of y, the nearest even value of n is used. The result is always exact.

`sin(x, /)`
Return the sine of x (measured in radians).

`sinh(x, /)`
Return the hyperbolic sine of x.

`sqrt(x, /)`
Return the square root of x.

`sumprod(p, q, /)`
Return the sum of products of values from two iterables p and q.

Roughly equivalent to:

```
sum(map(operator.mul, p, q, strict=True))
```

For float and mixed int/float inputs, the intermediate products and sums are computed with extended precision.

`tan(x, /)`
Return the tangent of x (measured in radians).

`tanh(x, /)`
Return the hyperbolic tangent of x.

`trunc(x, /)`
Truncates the Real x to the nearest Integral toward 0.

Uses the `__trunc__` magic method.

`ulp(x, /)`
Return the value of the least significant bit of the float x.

DATA

```
e = 2.718281828459045
inf = inf
nan = nan
pi = 3.141592653589793
tau = 6.283185307179586
```

```
FILE
(built-in)
```

You are now leaving help and returning to the Python interpreter.
If you want to ask for help on a particular object directly from the interpreter, you can type "help(object)". Executing "help('string')" has the same effect as typing a particular string at the help> prompt.

```
In [22]: from math import *
```

User Input function in Python

1. Input() in int

```
In [23]: x = input()
x
```

```
Out[23]: '5'
```

```
In [24]: x = input()
y = input()
z = x + y
print(z)  # Console is waiting for user to enter input
# Also if you work in idle
```

```
106
```

```
In [25]: print(type(x))
```

```
<class 'str'>
```

```
In [26]: x = int(input())  # Converts the string to int
y = int(input())
z = x + y
print(z)
```

```
10
```

```
In [27]: x1 = input('Enter the 1st number')  # Whenever you works in input function it a
y1 = input('Enter the 2nd number')  # It won't understand as arithmetic operato
z1 = x1 + y1
print(z1)
```

```
1819
```

```
In [28]: type(x1)
type(y1)
```

```
Out[28]: str
```

```
In [29]: x1 = input('Enter the 1st number')
         a1 = int(x1)
         y1 = input('Enter the 2nd number')
         b1 = int(y1)
         z1 = a1 + b1
         print(z1)
```

68

```
In [30]: type(a1)
         type(b1)
```

Out[30]: int

```
In [31]: x2 = int(input('Enter the 1st number')) # Here we're reducing some code lines a
         y2 = int(input('Enter the 2nd number'))
         z2 = x2 + y2
         print(z2)
```

90

2. Input() in String

```
In [32]: ch = input('Enter a char')
         print(ch)
```

Hello Team

```
In [33]: ch = input('Enter a char')[0]
         print(ch)
```

H

```
In [34]: print(ch[1])
```

```
-----
IndexError                                Traceback (most recent call last)
Cell In[34], line 1
----> 1 print(ch[1])

IndexError: string index out of range
```

```
In [35]: ch = input('Enter a char')
         print(ch[1])
```

e

```
In [36]: ch = input('Enter a char')
         print(ch[-1])
```

g

```
In [37]: ch = input('Enter a char')[1:3]
         print(ch)
```

el

```
In [38]: ch = input('Enter a char')
         print(ch) # If you enter as 2 + 6 -1 ,we get output as 2 + 6 -1
```

2 + 6 - 1

3. Input() in Expressions (EVAL function using input)

```
In [39]: result = input('Expression')  
print(result)
```

2 + 3 + 5 - 2

```
In [40]: result = eval(input('Enter an expression')) # 'eval' is used for expression  
print(result)
```

8

```
In [ ]:
```